Linear Search

This technique used to find elements in list of array.

In this method each element checked one by one from the begining until the desired element is found or the end of the list is reached.

Time complexity
Best case : O(1)
Average case : O(n)
Worst case : O(n)

Space complexity
O(1)
because constant amount of extra memory is used.

```
#include<stdio.h>
int main(){
   int roll[10], n, i, key, found = 0;
printf("enter the no of students:");
   scanf("%d", &n);
printf("enter the roll no:\n");
   for(i=0; i<n; i++){
      scanf("%d", &roll[i]);}
printf("Enter the roll no to search:");
   scanf("%d", &key);
   for(i=0; i<n; i++){
      if(roll[i]==key){
         found=1;
         printf("found at index %d\n", i);
   if(found==0){
```

main.c

Share Run

```c
#include <stdio.h>
int main() {
    int  roll[10] ,n, i, key, found = 0;
   // Input number of students
    printf("Enter the number of students: ");
    scanf("%d", &n);
  // Input roll numbers
    printf("Enter the roll numbers:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &roll[i]);}
  // Input roll number to search
    printf("Enter the roll number to search: ");
    scanf("%d", &key);
  // Linear Search
    for(i = 0; i < n; i++) {               //linear sesarch
        if(roll[i] == key) {
            found = 1;
            printf("Found at index %d\n",i);
            break;
       if(found == 0) {                          //if not fpound
        printf("roll noumber is not find");}
     return 0;}
```

Output

```
Enter the number of students: 05
Enter the roll numbers:
101 105 113 110 130
Enter the roll number to search: 110
Found at index 3


=== Code Execution Successful ===
```

```
printf("you no is not find");}
return 0;}
```

Bineary Search

This technique is used to find an element in sorted array. It repeatedly divides the search interval into two halves until the desired element is found or the search interval becomes empty.

Time complexity is O(logn) in worst and average cases. O(1) in the best case

```
#include<stdio.h>
int main(){
  int arr[10], n, i, j, key;
  printf("Enter the no of books :");
  scanf("%d", &n);
  printf("Enter %d book serial no :\n", n);
  for(i=0; i<n; i++){
    scanf("%d", &arr[i]);}

For(i=1; i<n; i++){
    key = arr[i];
    j--;}
  arr[j+1] = key;}
```

Insertion sort code

main.c

Share Run

```c
#include <stdio.h>
int main() {
    int  arr[10],n, target, i;
    printf("Enter the no of prducts ID :-");
    scanf("%d", &n);
    printf("enter the sorted product ID:-");
    for(i = 0; i < n; i++) {
    scanf("%d", &arr[i]);}
    printf("enter the product ID to search :-");
    scanf("%d", &target);
   int low = 0, high = n - 1, mid;
    int found = 0;
    while(low <= high) {                          //Bineary search
        mid = (low + high) / 2;
    if(arr[mid] == target) {
            found = 1;
            break;}
        else if(arr[mid] < target) {
            low = mid + 1;}
      else {
            high = mid - 1;}}
   if(found)
        printf("Product Available");
    else
        printf("Product Not Available");
  return 0;
```

Output

```
Enter the no of prducts ID :-06
enter the sorted product ID:-10 20 30 40 50 60
enter the product ID to search :-40
Product Available

=== Code Execution Successful ===
```

```
printf("sorted book serial no are :-\n");
    for (i=0 ; i< n; i++){
    printf ("%d", arr[i]); }
      return 0;
```

Bubble sort

It is simple sorting algorithm that repeatedly compares two adjacents elements and swaps them if they are in the wrong order. This process continous until the array is completely sorted.

```
#include <stdio.h>
 int main(){
   int arr[10], n, i, j, temp;
 printf("enter the no of exam score:");
   scanf("%d", &n);
 printf("enter %d exam score:\n");
    for(i=0; i<n; i++){
      scanf("%d", &arr[i]); }
  For( i=0 ; i<n-1 ; i++){
   For(j=0; j<n-i-1; j++){
        if (arr[j] > arr[j+1]){
          temp = arr[j];
         arr[j] = arr[j+1];
```

main.c

Share Run

```c
#include <stdio.h>

int main(){
    int arr[10], n, i, j, temp;
    printf("Enter the number of exam scores: ");
    scanf("%d", &n);
    printf("Enter %d exam scores:\n", n);
    for(i = 0; i < n; i++){
      scanf("%d", &arr[i]);}
    for(i = 0; i < n - 1; i++){
    for(j = 0; j < n - i - 1; j++){
          if(arr[j] > arr[j + 1]){
              temp = arr[j];
               arr[j] = arr[j + 1];
               arr[j + 1] = temp;
           }}}
    printf("Sorted exam scores in ascending order are:\n");
     for(i = 0; i < n; i++){
    printf("%d ", arr[i]);}
    return 0;}

```

Output

```
Enter the number of exam scores: 05
Enter 5 exam scores:
89 45 78 12 99
Sorted exam scores in ascending order are:
12 45 78 89 99

=== Code Execution Successful ===
```

```
arr[j+1] = temp; }}

printf("Sorted exam score in ascending
    order are :\n");
for(i=0; i<n; i++){
    printf("%d", arr[i]); }
    return 0;
```

Insertion sort

Insertion sort is a simple algorithm that builds the sorted array one elements at a time. It takes each elements and inserts it into its coorect position in the already sorted part of the array.

linear search code

```
#include <stdio.h>
int main(){
    int arr[10], n, target, i;
    printf("Enter the no of product ID:-");
    scanf("%d", &n);
    printf("enter the sorted product ID:-");
    for(i=0; i<n; i++){
        scanf("%d", &arr[i]); }
    printf("enter the product ID to search:-");
    scanf("%d", &target);
    int low=0, high=n-1, mid;
```

main.c

Share Run

```c
#include <stdio.h>
int main(){
 int  arr[10],n, i, j, key;
  printf("Enter the number of books: ");
 scanf("%d", &n);
    printf("Enter %d book serial numbers:\n", n);
    for(i = 0; i < n; i++){
     scanf("%d", &arr[i]);}
    for(i = 1; i < n; i++){
        key = arr[i];
        j = i - 1;
        while(j >= 0 && arr[j] > key){
            arr[j + 1] = arr[j];
            j--;}
      arr[j + 1] = key;}
    printf("Sorted book serial numbers are:\n");
     for(i = 0; i < n; i++){
      printf("%d ", arr[i]);}
      return 0;
}
```

Output

```
Enter the number of books: 05
Enter 5 book serial numbers:
55 23 87 11 34
Sorted book serial numbers are:
11 23 34 55 87

=== Code Execution Successful ===
```

```
int found = 0;
while (low <= high) {
    mid = (low + high) / 2;
if (arr[mid] == target) {
        found = 1;
        break;
else if (arr[mid] < target) {
        low = mid + 1; }
    else { high = mid - 1; }}
if (found)
    printf ("product Available");
else
    printf ("product Not Available");
    return 0;
```

Selection Sort

Selection Sort is a simple sorting algorithm that repeatedly selects the smallest elements from the unsorted part of the array and place it at the beginning. this process continous until the entire array is sorted.

```
#include<stdio.h>
int main(){
  int arr[10], n, i, j, minIndex, temp;
  printf("enter no of players = ");
    scanf("%d", &n);
  printf("enter %d player score \n", n);
  for(i=0; i<n; i++){
     scanf("%d", &arr[i]);}
  for(i=0; i<n+1; i++){
     if(arr[j] < arr[minIndex]){
        minIndex = j;}}
     temp = arr[i];
     arr[i] = arr[minIndex];
     arr[minIndex] = temp;}
  printf("sorted player score = ");
    for(i=0; i<n; i++){
      printf("%d", arr[i]);}
        return 0;}
```

ain.c

Share Run

```c
#include <stdio.h>

int main(){
int arr[10], n, i, j, minIndex, temp;
    printf("Enter the number of players: ");
    scanf("%d",&n);
    printf("Enter %d player scores:\n", n);
   for(i = 0; i < n; i++){
    scanf("%d", &arr[i]);}
    for(i = 0; i < n - 1; i++){
    minIndex = i;
     for(j = i + 1; j < n; j++){
         if(arr[j] < arr[minIndex]){
            minIndex = j;
            }}
        temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
    printf("Sorted player scores are:\n");
    for(i = 0; i < n; i++){
     printf("%d ", arr[i]);}
     return 0;
}
```

Output

```
Enter the number of players: 05
Enter 5 player scores:
70 20 90 10 40
Sorted player scores are:
10 20 40 70 90

=== Code Execution Successful ===
```